



Plan your trip with Kayak

Projet

Créer une application qui fait une recommandation de lieux où partir en vacances, à partir de données météo et des hôtels environnants.

L'application sera, pour ce projet uniquement, focalisée sur 35 villes en France uniquement.

Plan du notebook

1. Extraction des données météo
2. Extraction des hôtels par ville
3. Fusion des données et enregistrement dans un S3
4. Database
5. Les deux cartes
6. Conclusion

```
In [1]: import os
import sys
import json
import time
import requests
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import boto3
from dotenv import load_dotenv
from sqlalchemy import create_engine, text
```

```
# Lecture du fichier .env (clés d'API et mots de passe)
load_dotenv()
```

Out[1]: True

Partie 1 : Extraction des données météo

Coordonnées GPS

```
In [2]: # Récupération des coordonnées GPS des villes à partir de OpenStreetMap

pays = 'France'

villes_brutes = ["Mont Saint Michel", "St Malo", "Bayeux", "Le Havre", "Rouen",
                 "Paris", "Amiens", "Lille", "Strasbourg", "Chateau du Haut Koen",
                 "Colmar", "Eguisheim", "Besancon", "Dijon", "Annecy",
                 "Grenoble", "Lyon", "Gorges du Verdon", "Bormes les Mimosas", "
                 "Marseille", "Aix en Provence", "Avignon", "Uzes", "Nimes",
                 "Aigues Mortes", "Saintes Maries de la mer", "Collioure", "Carc",
                 "Toulouse", "Montauban", "Biarritz", "Bayonne", "La Rochelle"]

headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
url_base = 'https://nominatim.openstreetmap.org/search?'

liste_coordonnees = []
for ville in villes_brutes:
    ville_recherche = ville.replace(' ', '%20')
    url_total = f"{url_base}q={ville_recherche}+{pays}&format=geojson"

    r = requests.get(url_total, headers=headers)
    r = r.json()
    #print('status code :', r.status_code)
    coordonnees = r['features'][0]['geometry']['coordinates'] # [longitude, La

    liste_coordonnees.append({'ville': ville,
                              'longitude': coordonnees[0],
                              'latitude': coordonnees[1]})

    time.sleep(1)

df_villes = pd.DataFrame(liste_coordonnees)
df_villes['ville_id'] = df_villes.index + 1 # identifiant unique de chaque v
df_villes.to_csv('coordonnees_villes.csv', index=False)
df_villes.head()
```

Out[2]:

	ville	longitude	latitude	ville_id
0	Mont Saint Michel	-1.511460	48.635954	1
1	St Malo	-2.026041	48.649518	2
2	Bayeux	-0.702474	49.276462	3
3	Le Havre	0.107973	49.493898	4
4	Rouen	1.093966	49.440459	5

```
In [3]: df_villes.isna().sum().sum()
```

```
Out[3]: np.int64(0)
```

```
In [4]: # Vérification rapide : Les 35 villes sont-elles au bon endroit ?
fig = px.scatter_map(df_villes, lat='latitude', lon='longitude', text='ville',
                    center={'lat': 46.6, 'lon': 2.4}, zoom=4.5, height=500)
fig.show()
```

Données météo des villes

```
In [5]: # Récupération des infos météo à l'aide d'OpenWeatherMap (pour une ville)
cle_meteo = os.environ['OWM_KEY']

# Test sur la première ville de la liste
ville_test = df_villes.iloc[0]
url_meteo = f"https://api.openweathermap.org/data/2.5/forecast?lat={ville_test['lat']}&lon={ville_test['lon']}&appid={cle_meteo}"

r = requests.get(url_meteo)
print('status code :', r.status_code)
r.json().keys()
```

status code : 200

```
Out[5]: dict_keys(['cod', 'message', 'cnt', 'list', 'city'])
```

```
In [6]: previsions_test = r.json()['list']
print("Nombre de prévisions :", len(previsions_test))
previsions_test[0]
```

Nombre de prévisions : 40

```
Out[6]: {'dt': 1791234000,
        'main': {'temp': 15.34,
                  'feels_like': 15.25,
                  'temp_min': 15.34,
                  'temp_max': 15.34,
                  'pressure': 1022,
                  'sea_level': 1022,
                  'grnd_level': 1021,
                  'humidity': 89,
                  'temp_kf': 0,
                  'dew_point': 13.54},
        'weather': [{'id': 802,
                      'main': 'Clouds',
                      'description': 'scattered clouds',
                      'icon': '03n'}],
        'clouds': {'all': 33},
        'wind': {'speed': 1.95, 'deg': 66, 'gust': 3.11},
        'visibility': 10000,
        'pop': 0,
        'sys': {'pod': 'n'},
        'dt_txt': '2026-10-05 21:00:00'}
```

```
In [7]: # récupération des données météo brutes pour l'ensemble des villes
# La consigne cite l'API One Call (prévisions sur 7 jours). On utilise ici l'API
# prévisions sur 5 jours, une toutes les 3 heures (40 prévisions par ville), ce
liste_previsions = []
for i in range(len(df_villes)):
    ville = df_villes.loc[i, 'ville']
    latitude = df_villes.loc[i, 'latitude']
```

```

longitude = df_villes.loc[i, 'longitude']

url_meteo = f"https://api.openweathermap.org/data/2.5/forecast?lat={latitude}"
r = requests.get(url_meteo)
#print(f"{ville} : status code {r.status_code}")
previsions = r.json()['list']

# Une ligne par prévision (40 par ville)
for p in previsions:
    liste_previsions.append({'ville_id': df_villes.loc[i, 'ville_id'],
                             'ville': ville,
                             'date_heure': p['dt_txt'],
                             'temp': p['main']['temp'],
                             'temp_min': p['main']['temp_min'],
                             'temp_max': p['main']['temp_max'],
                             'temp_ressentie': p['main']['feels_like'],
                             'humidite': p['main']['humidity'],
                             'temps' : p['weather'][0]['main'],
                             'nuages': p['clouds']['all'],
                             'pop': p['pop'],
                             'pluie_mm': p.get('rain', {}).get('3h', 0)})

#print(f"prévisions de {ville} récupérées")

df_prev = pd.DataFrame(liste_previsions)
df_prev.to_csv('previsions_brutes.csv', index=False)
print(df_prev.shape)
df_prev.head()

```

(1400, 12)

Out[7]:

	ville_id	ville	date_heure	temp	temp_min	temp_max	temp_ressentie	humidite
0	1	Mont Saint Michel	2026-10-05 21:00:00	15.34	15.34	15.34	15.25	89
1	1	Mont Saint Michel	2026-10-06 00:00:00	14.96	14.19	14.96	14.86	90
2	1	Mont Saint Michel	2026-10-06 03:00:00	14.51	14.09	14.51	14.31	88
3	1	Mont Saint Michel	2026-10-06 06:00:00	13.30	13.30	13.30	12.96	87
4	1	Mont Saint Michel	2026-10-06 09:00:00	17.99	17.99	17.99	17.70	71

Résumé météo par ville

À partir de df_prev, on calcule pour chaque ville :

- temp_ressentie_moy : température moyenne ressentie
- nuages_moy : couverture nuageuse moyenne

- proba_pluie_max : probabilité de pluie maximale
- pluie_totale_mm : volume de pluie attendu sur les 5 jours (somme des rain.3h, 0 quand il ne pleut pas)

On souhaite les villes avec la meilleure météo. On définit la meilleure météo comme celle où il ne pleut pas et où il ne fait pas trop froid. On va donc calculer un score de météo à partir d'un score de pluie et de la température ressentie

- score de pluie de 0 (très sec) à 100 (très pluvieux) = $100 \times (0.6 \times \text{pluie_totale_mm} / \text{pluie_totale_mm maximale des villes} + 0.2 \times \text{proba_pluie_max} + 0.2 \times \text{nuages_moy} / 100)$
- score de température = $100 \times |\text{temp_ressentie_moy} - 20| / 20$: il vaut 0 à 20 °C ressentis et augmente en fonction de l'écart par rapport à 20 °C
- score de météo = $0.7 \times \text{score de pluie} + 0.3 \times \text{score de température}$ (plus il est bas, meilleure est la météo)

```
In [8]: liste_meteo = []
for i in range(len(df_villes)):
    ville = df_villes.loc[i, 'ville']

    # Les 40 prévisions de cette ville
    df_ville = df_prev[df_prev['ville'] == ville]

    liste_meteo.append({'ville_id': df_villes.loc[i, 'ville_id'],
                        'ville': ville,
                        'latitude': df_villes.loc[i, 'latitude'],
                        'longitude': df_villes.loc[i, 'longitude'],
                        'temp_ressentie_moy': df_ville['temp_ressentie'].mean(),
                        'nuages_moy': df_ville['nuages'].mean(),
                        'proba_pluie_max': df_ville['pop'].max(),
                        'pluie_totale_mm': df_ville['pluie_mm'].sum()})

df_meteo = pd.DataFrame(liste_meteo).round(2)

# Score de pluie
df_meteo['score_pluie'] = 100 * (0.6 * df_meteo['pluie_totale_mm'] / df_meteo['p
                        + 0.2 * df_meteo['proba_pluie_max']
                        + 0.2 * df_meteo['nuages_moy'] / 100)
df_meteo['score_pluie'] = df_meteo['score_pluie'].round(1)

# Score de température
ecart_temp = (df_meteo['temp_ressentie_moy'] - 20).abs()
df_meteo['score_temperature'] = (100 * ecart_temp / 20).round(1)

# Score de météo
df_meteo['score_meteo'] = (0.7 * df_meteo['score_pluie'] + 0.3 * df_meteo['score

print(df_meteo.shape)
df_meteo.head()
```

(35, 11)

Out[8]:

	ville_id	ville	latitude	longitude	temp_ressentie_moy	nuages_moy	proba_pluie_r
--	----------	-------	----------	-----------	--------------------	------------	---------------

0	1	Mont Saint Michel	48.64	-1.51	14.78	70.05
1	2	St Malo	48.65	-2.03	15.60	70.72
2	3	Bayeux	49.28	-0.70	13.74	70.45
3	4	Le Havre	49.49	0.11	15.26	74.12
4	5	Rouen	49.44	1.09	13.52	68.68

```
In [9]: df_meteo = df_meteo.sort_values(by=['score_meteo', 'temp_ressentie_moy'], ascend
df_meteo = df_meteo.reset_index(drop=True)
df_meteo['rang_meteo'] = df_meteo.index + 1
df_meteo['top5_meteo'] = df_meteo['rang_meteo'] <= 5

df_meteo.to_csv('meteo_villes.csv', index=False)

meilleure_ville = df_meteo.loc[0, 'ville']
print("La destination avec le plus beau temps est :", meilleure_ville)
df_meteo[df_meteo['top5_meteo']]
```

La destination avec le plus beau temps est : Collioure

Out[9]:

	ville_id	ville	latitude	longitude	temp_ressentie_moy	nuages_moy	proba_pluie
--	----------	-------	----------	-----------	--------------------	------------	-------------

0	28	Collioure	42.53	3.08	17.11	51.80
1	4	Le Havre	49.49	0.11	15.26	74.12
2	35	La Rochelle	46.16	-1.15	16.90	63.82
3	6	Paris	48.85	2.35	14.21	64.47
4	11	Colmar	48.08	7.36	13.34	50.35

Partie 2 : Extraction des hôtels par ville

La consigne suggère de scraper Booking.com. On utilise Scrapy : une classe Spider, lancée avec un CrawlerProcess, qui enregistre le résultat dans un fichier json.

Le spider lit, sur la page de résultats de la ville, le nom, l'url et la note de chaque hôtel. Pour commencer, une seule ville (variable `ville_scraping`).

```
In [10]: ville_scraping = 'La Rochelle' # ville traitée pour les hôtels pur l'essai
date_arrivee = '2026-10-17' # une date dans le futur
date_depart = '2026-10-18'

user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
```

```
In [11]: # Lancement du spider
!{sys.executable} scrapy_booking.py
```

Status code : 202 | taille de la page : 3962 octets

Aucun hotel trouve : Booking.com renvoie une page de verification anti-robot

```
In [12]: # Lecture du résultat
if os.path.exists('hotels_liste.json'):
    with open('hotels_liste.json', encoding='utf-8') as file:
        liste_hotels = json.load(file)
else:
    liste_hotels = []

df_liste = pd.DataFrame(liste_hotels)
print("Nombre d'hôtels récupérés :", len(df_liste))
if len(df_liste) == 0:
    print("Aucun hôtel récupéré : Booking.com a bloqué le scraping (status code",
df_liste.head()
```

Nombre d'hôtels récupérés : 0

Aucun hôtel récupéré : Booking.com a bloqué le scraping (status code 202, voir ci-dessus)

Out[12]: —

Résultat : Booking bloque le scraping

Booking a répondu avec le code 202 et une page de seulement 4 Ko. Aucun hôtel n'a été récupéré, la requête a été "acceptée sans être traitée totalement". Il s'agit probablement d'une protection anti bot.

Ce que l'on fait pour la suite : on enregistre à la main une page de résultats par ville (booking_la_rochelle.html, booking_colmar.html, ...) et on les scrape en local. La liste des villes est donnée manuellement au spider.

```
In [13]: # Lancement du spider sur les pages enregistrées à la main (une par ville)
!{sys.executable} scrapy_booking_local.py
```

Page lue : booking_la_rochelle.html
Taille de la page : 1580855 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_collioure.html
Taille de la page : 1547894 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_strasbourg.html
Taille de la page : 1605930 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_dijon.html
Taille de la page : 1574503 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_colmar.html
Taille de la page : 1573864 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_le_havre.html
Taille de la page : 1570413 octets

Nombre d'hôtels trouvés : 25

Page lue : booking_paris.html
Taille de la page : 1601824 octets

Nombre d'hôtels trouvés : 25

```
In [14]: # Lecture du résultat
with open('hotels_booking.json', encoding='utf-8') as file:
    liste_booking = json.load(file)

df_booking = pd.DataFrame(liste_booking)
print("Nombre d'hôtels récupérés :", len(df_booking))
df_booking.head()
```

Nombre d'hôtels récupérés : 175

Out[14]:

	ville	nom	url	note	nb_avis	e
0	La Rochelle	Hôtel La Fabrique	https://www.booking.com/hotel/fr/la-fabrique-l...	8.6	2319	
1	La Rochelle	Lagrange Apart'Hotel l'Escale Marine	https://www.booking.com/hotel/fr/l-escale-mari...	7.7	5638	
2	La Rochelle	Le Saint-Nicolas	https://www.booking.com/hotel/fr/comforthotels...	9.1	3397	
3	La Rochelle	Maisons du Monde Hôtel & Suites - La Rochelle ...	https://www.booking.com/hotel/fr/inter-saint-j...	8.6	3286	
4	La Rochelle	BW Plus Central Park Hotel & Spa	https://www.booking.com/hotel/fr/central-park-...	8.9	1857	

In [15]:

```
# Nettoyage de la liste des hôtels

df_hotels = df_booking.copy()
print("Hôtels récupérés :", len(df_hotels))

df_hotels = df_hotels.dropna(subset=['nom', 'latitude', 'longitude'])
df_hotels = df_hotels.drop_duplicates(subset='url')
df_hotels = df_hotels.reset_index(drop=True)
df_hotels['hotel_id'] = df_hotels.index + 1

df_hotels = df_hotels[['hotel_id', 'ville', 'nom', 'url', 'note', 'nb_avis', 'et

print("Hôtels après nettoyage :", len(df_hotels))
print("dont avec un nombre d'étoiles :", df_hotels['etoiles'].notna().sum())
df_hotels.head()
```

Hôtels récupérés : 175

Hôtels après nettoyage : 175

dont avec un nombre d'étoiles : 172

Out[15]:

	hotel_id	ville	nom	url	note	r
0	1	La Rochelle	Hôtel La Fabrique	https://www.booking.com/hotel/fr/la-fabrique-l...	8.6	
1	2	La Rochelle	Lagrange Apart'Hotel l'Escale Marine	https://www.booking.com/hotel/fr/l-escale-mari...	7.7	
2	3	La Rochelle	Le Saint-Nicolas	https://www.booking.com/hotel/fr/comforthotels...	9.1	
3	4	La Rochelle	Maisons du Monde Hôtel & Suites - La Rochelle ...	https://www.booking.com/hotel/fr/inter-saint-j...	8.6	
4	5	La Rochelle	BW Plus Central Park Hotel & Spa	https://www.booking.com/hotel/fr/central-park-...	8.9	

Partie 3 : Fusion des données et enregistrement dans un S3

```
In [16]: # Fusion des deux dataframes

# On ne garde que les villes du Top-5 météo : La fusion garde seulement les vill
df_final = pd.merge(df_hotels, df_meteo[df_meteo['top5_meteo']], on='ville', suf

df_final.to_csv('kayak_data.csv', index=False)
df_final.shape
```

Out[16]: (125, 23)

Datalake sur S3

```
In [17]: AWS_KEY = os.environ['AWS_KEY']
AWS_SECRET_KEY = os.environ['AWS_SECRET_KEY']

session = boto3.Session(
    aws_access_key_id=AWS_KEY,
```

```
aws_secret_access_key=AWS_SECRET_KEY,
region_name="eu-west-3")

s3 = session.resource("s3")
```

```
In [18]: nom_bucket = "shy-certif-cdsd"

# bucket = s3.create_bucket(
#     Bucket=nom_bucket,
#     CreateBucketConfiguration={"LocationConstraint": "eu-west-3"})

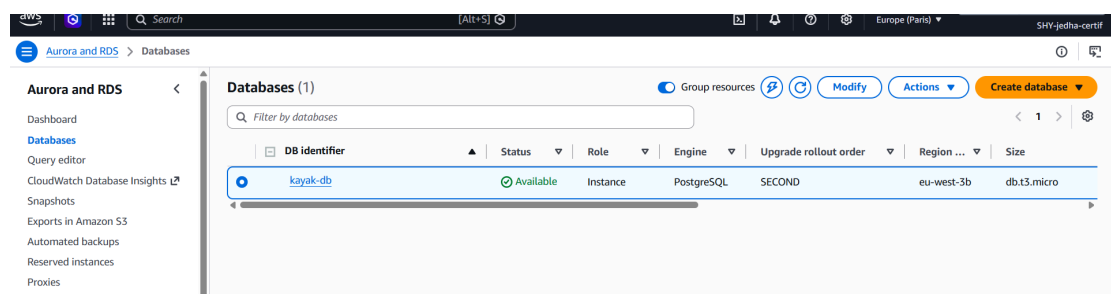
bucket = s3.Bucket(nom_bucket)
```

```
In [19]: # Dépôt du fichier csv dans le bucket
bucket.upload_file('kayak_data.csv', 'kayak_data.csv')

for objet in bucket.objects.all():
    print(objet)
```

```
s3.ObjectSummary(bucket_name='shy-certif-cdsd', key='kayak_data.csv')
```

Partie 4 : Database



```
In [20]: DB_USER = os.environ['DB_USER']
DB_PASSWORD = os.environ['DB_PASSWORD']
DB_HOST = os.environ['DB_HOST']
DB_PORT = os.environ['DB_PORT']
DB_NAME = os.environ['DB_NAME']

engine = create_engine(f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}")
conn = engine.connect()
```

```
In [21]: # EXTRACT : on télécharge le csv depuis S3
bucket.download_file('kayak_data.csv', 'kayak_data_depuis_s3.csv')
df_s3 = pd.read_csv('kayak_data_depuis_s3.csv')
print("Lignes extraites de S3 :", len(df_s3))

# TRANSFORM : on nettoie
df_propre = df_s3.drop_duplicates(subset='url')
df_propre = df_propre.dropna(subset=['nom', 'latitude', 'longitude'])
print("Lignes après nettoyage :", len(df_propre))

# LOAD : on charge le tableau dans la base SQL
df_propre.to_sql('hotels_meteo', engine, index=False, if_exists='replace')
```

Lignes extraites de S3 : 125
Lignes après nettoyage : 125

```
Out[21]: 125
```

Interrogation de la base

```
In [22]: df_villes_db = pd.read_sql_query(text("SELECT ville, COUNT(*) FROM hotels_meteo"), conn)
df_villes_db
```

Out[22]:

	ville	count
0	La Rochelle	25
1	Collioure	25
2	Le Havre	25
3	Paris	25
4	Colmar	25

```
In [23]: df_db = pd.read_sql_query(text("SELECT * FROM hotels_meteo"), conn)
df_db.head()
```

Out[23]:

	hotel_id	ville	nom	url	note
0	1	La Rochelle	Hôtel La Fabrique	https://www.booking.com/hotel/fr/la-fabrique-l...	8.6
1	2	La Rochelle	Lagrange Apart'Hotel l'Escale Marine	https://www.booking.com/hotel/fr/l-escale-mari...	7.7
2	3	La Rochelle	Le Saint-Nicolas	https://www.booking.com/hotel/fr/comforthotels...	9.1
3	4	La Rochelle	Maisons du Monde Hôtel & Suites - La Rochelle ...	https://www.booking.com/hotel/fr/inter-saint-j...	8.6
4	5	La Rochelle	BW Plus Central Park Hotel & Spa	https://www.booking.com/hotel/fr/central-park-...	8.9

5 rows × 6 columns



Partie 5 : Les deux cartes

- Carte 1 : le Top-5 des destinations avec le plus beau temps (données météo).
- Carte 2 : le Top-20 des hôtels de chaque ville du Top-5, avec un menu déroulant pour choisir la ville (données Booking).

Les deux cartes sont construites avec des requêtes SQL sur la base (RDS).

```
In [24]: # Les 5 villes du Top-5 : une ligne par ville, lue dans la base SQL
requete_villes = text("""
    SELECT DISTINCT ville, latitude_ville AS latitude, longitude_ville AS longit
           rang_meteo, score_meteo, temp_ressentie_moy, pluie_totale_mm
    FROM hotels_meteo
    ORDER BY rang_meteo""")
df_top5_db = pd.read_sql_query(requete_villes, conn)

fig = px.scatter_map(
    df_top5_db,
    lat='latitude',
    lon='longitude',
    text='ville',
    hover_name='ville',
    hover_data={'rang_meteo': True, 'score_meteo': True, 'temp_ressentie_moy': T
               'latitude': False, 'longitude': False},
    color='score_meteo',
    labels={'score_meteo': 'Score météo (bas = beau temps)'},
    center={'lat': 46.6, 'lon': 2.4},
    zoom=4.5,
    height=500,
    title='Top-5 des destinations avec le plus beau temps (5 prochains jours)')

fig.update_layout(margin={"r": 0, "t": 40, "l": 0, "b": 0})
fig.update_traces(marker=dict(size=20))
fig.show()
```

```
In [25]: # Les 20 meilleurs hôtels de chaque ville du Top-5, lus dans la base SQL, avec u
# On classe par note (à note égale : celui qui a le plus d'avis)
villes_top5 = list(df_top5_db['ville'])      # la ville avec le plus beau temps

requete = text("""
    SELECT * FROM hotels_meteo
    WHERE ville = :ville
    ORDER BY note DESC NULLS LAST, nb_avis DESC NULLS LAST
    LIMIT 20""")

fig = go.Figure()
liste_boutons = []

for i in range(len(villes_top5)):
    ville = villes_top5[i]
    top20 = pd.read_sql_query(requete, conn, params={'ville': ville})

    top20['description_courte'] = top20['description'].fillna('').str[:150] + '

    fig.add_trace(go.Scattermap(
```

```

lat=top20['latitude'],
lon=top20['longitude'],
mode='markers',
marker=dict(size=12, color=top20['note'], colorscale='Plasma', cmin=7, c
              colorbar=dict(title='Note')),      # échelle de couleurs fixé
text=top20['nom'],
customdata=top20[['note', 'nb_avis', 'etoiles', 'quartier', 'description
hovertemplate=( "%{text}<br>Note : %{customdata[0]}<br>Avis : %{customdat
                 "<br>Quartier : %{customdata[3]}<br>%{customdata[4]}<br>%

name=ville,
visible=(i == 0)))

lat_ville = top20['latitude_ville'].iloc[0]
lon_ville = top20['longitude_ville'].iloc[0]
visible = [j == i for j in range(len(villes_top5))]

liste_boutons.append(dict(
    label=ville,
    method='update',
    args=[{'visible': visible},
          {'map.center': {'lat': lat_ville, 'lon': lon_ville},
           'title.text': f"Top-20 des hôtels à {ville} (classés par note des

# Réglages de départ : carte centrée sur la première ville
premiere = df_top5_db.iloc[0]
fig.update_layout(
    map=dict(style='open-street-map',
             center={'lat': premiere['latitude'], 'lon': premiere['longitude']},
             zoom=13),
    title=f"Top-20 des hôtels à {villes_top5[0]} (classés par note des avis)",
    updatemenus=[dict(buttons=liste_boutons, direction='down',
                      x=0.01, xanchor='left', y=0.99, yanchor='top')],
    margin={"r": 0, "t": 40, "l": 0, "b": 0},
    height=600)
fig.show()

```

```

In [26]: # Fermeture de La connexion à La base
conn.close()

```

Partie 6 : Conclusion

- Data lake : kayak_data.csv est dans le bucket S3.
- Data warehouse : les mêmes données nettoyées sont dans la table hotels_meteo de la base PostgreSQL (RDS).
- Cartes : Top-5 des destinations (météo) et Top-20 des hôtels.

Ce qui diffère de la consigne : Booking.com bloque le scraping automatique (réponse 202, page de vérification anti-robot). On a donc enregistré à la main une page de résultats pour quelques villes du Top-5 météo (25 hôtels par page) et on les a scrapées en local avec Scrapy. (Si la ville avec la meilleure météo n'a pas été scrapée, cela pose problème)

Pour aller plus loin : traiter les 30 autres villes, avec une page Booking par ville ou en passant par une source de données autorisée (API de trip advisor par exemple).

